

Exam / Project

Deadline 20th of May

Please submit your code as a ZIP file and your presentation as a PDF on DigitalExamen.

You can work in teams of up to three. Everyone in a team needs to be able to present and explain the findings and code.

The oral exam will be in the form of a scientific presentation. You are supposed to present your findings in terms of strong and weak scaling and explain the parallelisation strategies and/or data layouts used. The presentations will be scheduled in the last two weeks of the semester. The presentation will be one per group.

There will be a total of 100 points. You need 50 points to pass.

Parallelisation of Matrix-Matrix Multiplications

Matrix-matrix multiplications are a common problem that appears in many codes aimed at HPC systems. This project aims to implement a matrix-matrix multiplication using different parallelisation paradigms. We will look at shared-memory systems with OpenMP. Distributed memory system with MPI. And accelerator (GPU) systems with CUDA.

$$\begin{matrix} \text{row } i & \left[\begin{array}{c} \text{---} \end{array} \right] & \times & \left[\begin{array}{c} \text{col } j \\ \text{---} \end{array} \right] & = & \left[\begin{array}{c} \text{---} \\ \text{---} \end{array} \right] \\ & A & & B & & C \\ & m \times n & & n \times p & & m \times p \end{matrix}$$
$$c_{ij} = \text{row } i \times \text{col } j = \sum_{k=1}^n a_{ik} b_{kj}$$

Your primary goal is to evaluate the **Strong** and **Weak scaling** properties of each of your implementations.

Please make sure that your codes are correct. Compare your matrix C to the one obtained by your serial code. To verify correctness, you can calculate a checksum (sum of all elements in C) or compare the norm of the difference between the serial and parallel matrices ($\|C_{\text{serial}} - C_{\text{parallel}}\|$).

I suggest starting with matrices of the size 1024^2 , which can later be increased to 8096^2 .

Task 0: Serial Implementation

[10 Points]

Create a reference program to verify correctness and measure baseline performance.

- Write a standard C program to multiply two $N \times N$ matrices.
- Implement the multiplication using the naive IJK loop order.
- Use a high-resolution timer like `clock_gettime` or `omp_get_wtime` to record the duration.

Task 1: OpenMP (Shared Memory)

[10 Points]

Parallelise the computation on a single multi-core CPU.

- Use `#pragma omp parallel` for to distribute the work.
- Explore different scheduling, collapse and other OpenMP options for maximum performance gains.
- Explore different loop orders, fx change the naive IJK to IKJ ordering.

Task 2: MPI (Distributed Memory)

[25 Points]

Parallelise your implementation using Message Parsing Interface (MPI).

- Devise a strategy to do your computations. Think about how to block your matrices or how to separate your matrices onto different cores.
- Distribute your matrices onto different cores, e.g., by using `MPI_Scatter`.
- Collect matrix C in the main process, fx using `MPI_Gather`.

Task 3: CUDA (GPU Acceleration)

[35 Points]

Offload the computation to a GPU accelerator.

- Write a CUDA kernel where each thread computes one element of the output matrix.
- Manage memory transfers between the host and the device.

Task 4: Analysis and Scaling

[20 Points]

Evaluate the performance and scaling of all implementations.

- Check the strong scaling of your matrix-multiplication codes.
- Check the weak scaling of your matrix-multiplication codes. (Keep in mind that naive matrix multiplications scale with N^3).

More information on strong and weak scaling can be found at www.kth.se/blogs/pdc/2018/11/scalability-strong-and-weak-scaling/ as well as in the lecture slides.